



Como funcionam as aplicações web?

Programação Web II

Como funcionam as aplicações web?

Programação Web II

Programação

Distribuída

Programação distribuída envolve a execução de diferentes partes de uma aplicação em máquinas distintas conectadas em rede. Essa abordagem permite a comunicação entre **clientes** e **servidores**, viabilizando uma diversidade abrangente de serviços e recursos.

- **Cliente:** Software ou dispositivo que realiza solicitações e consome serviços fornecidos por servidores.
- **Servidor:** Sistema ou serviço que processa solicitações, executa tarefas e retorna respostas apropriadas.

Como funcionam as aplicações web?

Programação Web II

Programação

Distribuída

Características

- **Distribuição de carga:** melhor desempenho e escalabilidade.
- **Tolerância a falhas:** operações continuam mesmo se um servidor falhar.
- **Localização remota:** acesso a serviços de qualquer lugar do mundo.

Exemplo

Ao acessar um site no navegador, este atua como cliente ao enviar uma solicitação ao servidor, que responde com a página web correspondente. Essa interação exemplifica como a programação distribuída é aplicada nas aplicações web modernas.

HTTP (*Hypertext Transfer Protocol*) é o principal protocolo de comunicação na web.

Ele especifica como solicitações e respostas devem ser estruturadas para garantir um intercâmbio de informações eficiente entre clientes e servidores.

Estrutura de uma requisição HTTP

```
GET /index.html HTTP/1.1  
Host: www.exemplo.com
```

1. **Método:** determina a ação desejada (ex.: GET, POST, PUT, DELETE).
2. **URL:** indica o recurso solicitado no servidor.
3. **Versão:** versão do protocolo utilizado.
4. **Cabeçalhos:** informações adicionais para a requisição (opcionais).

Estrutura de uma resposta HTTP

Considerando a requisição enviada anteriormente para exibir "index.html", vamos examinar uma possível resposta.

```
HTTP/1.1 200 OK
Content-Type: text/html

<html> ... conteúdo ... </html>
```

1. **Status Code:** indica o resultado do processamento (ex.: 200 OK, 404 Not Found).
2. **Cabeçalhos:** metadados sobre a resposta.
3. **Corpo:** conteúdo retornado pelo servidor.

Protocolo HTTP

Programação Web II

Métodos HTTP mais comuns

- **GET:** recupera informações de um recurso.
- **POST:** envia dados para criação de novos recursos.
- **PUT:** atualiza recursos existentes.
- **DELETE:** remove recursos especificados.
- **PATCH:** atualiza parte de um recurso existente.

Códigos de status comuns

- **2xx:** Sucesso (ex.: 200 OK, 201 Created).
- **3xx:** Redirecionamento (ex.: 301 Moved Permanently).
- **4xx:** Erros do cliente (ex.: 404 Not Found).
- **5xx:** Erros do servidor (ex.: 500 Internal Server Error).

REST (*Representational State Transfer*) é um estilo arquitetural para comunicação web, estabelecendo princípios para facilitar a interoperabilidade entre sistemas distribuídos.

Princípios REST

- **Stateless:** cada requisição é processada de forma independente.
- **Recursos identificáveis:** urls únicas representam recursos acessíveis.
- **Operações padronizadas:** baseado nos métodos http padronizados.

Breve explicação sobre Json

O que é JSON?

JSON (JavaScript Object Notation)

JSON é um formato utilizado para troca de dados entre:

- aplicações
- APIs
- cliente e servidor

Hoje, é o formato mais utilizado na comunicação web.

Exemplo de Json:

```
{  
  "nome": "Maria",  
  "idade": 20,  
  "cidade": "São Paulo"  
}
```

Como ler esse JSON?

Parte	Significado
nome	chave
"Maria"	valor
idade	chave
20	valor

Regras importantes:

- JSON usa { } para objetos
- JSON usa " " para textos
- Dados ficam em pares:
 - chave → valor

Analogia simples

JSON funciona como:

- uma ficha de cadastro
- um formulário
- um objeto com informações

JSON em APIs REST

Exemplo de resposta de uma API

Quando fazemos uma requisição: **GET /users/1**

A API pode responder:

```
{  
  "id": 1,  
  "nome": "Stephannie",  
  "email": "stephannie@email.com",  
  "ativo": true  
}
```

Tipos de dados em JSON:

Tipo	Exemplo
Texto	"Stephannie"
Número	1
Texto	"stephannie@email.com"
Boolean	true

Exemplo de lista

```
{  
  "produtos": [  
    "Mouse",  
    "Teclado",  
    "Monitor"  
  ]  
}
```

APIs normalmente:

- recebem JSON
- processam JSON
- devolvem JSON

Por isso entender JSON é fundamental no desenvolvimento web.

Exemplos de Boas Práticas para Definição de URIs

Boas Práticas

1. **Clareza e Coerência:** Use nomes descritivos e coesos para recursos.

```
GET /users          # Obtém a lista de todos os usuários
GET /users/{id}     # Obtém detalhes de um usuário específico
POST /users         # Cria um usuário
PUT /users/{id}     # Atualiza completamente um usuário específico
PATCH /users/{id}  # Atualiza parcialmente um usuário
DELETE /users/{id}  # Remove um usuário
```


Boas Práticas

2. Identificação Hierárquica de Sub-recursos:

```
GET /users/{id}/orders          # Obtém todos os pedidos de um usuário específico  
GET /users/{id}/orders/{orderId} # Obtém detalhes de um pedido específico de um usuário
```

API REST

Exemplos de Boas Práticas para Definição de URIs

Boas Práticas

3. **Uso de Pluralidade para Representação de Coleções:**

GET /products	# Correto: usa plural para coleção de produtos
GET /products/{productId}	# Obtém um único produto específico

Boas Práticas

4. **Evitar Verbos na URI (Ação é Definida pelo Método HTTP):**

POST `/books`

Correto: ação de criação implícita no POST

Boas Práticas

5. **Utilização de Parâmetros para Filtragem (Query Parameters):**

```
GET /books?author=John           # Pesquisa livros específicos pelo autor
```

```
GET /books?genre=fiction&year=2021 # Filtros combinados para melhor controle
```


Evite na construção de
uma URI

API REST

Exemplos do que não fazer sobre uma URI

Evite

1. URI Excessivamente Descritiva:

GET /getAllUsers	# Errado: verbo desnecessário na URI
DELETE /removeUser/{id}	# Errado: verbo redundante, ação é implícita no DELETE

2. Falta de Clareza ou Estrutura:

GET /users/123/order/456	# Errado: inconsistente (singular/plural misturados)
--------------------------	---

3. Uso de Verbos na Identificação de Recursos:

/createUser	# Errado: a ação deve ser definida pelo método HTTP
/deleteProduct/{productId}	# Errado pelo mesmo motivo

API REST

Exemplos do que não fazer sobre uma URI

Evite

4. Incluindo Filtragem como Parte da URI (Evitar Combinações Múltiplas):

```
GET /products/fiction/2021          # Errado: deve usar parâmetros de consulta
```

5. Uso de IDs Ambíguos ou Não Definidos:

```
GET /items/123XYZ890ABC             # Errado: dificuldade de compreensão do recurso
```

Esses exemplos mostram como URIs devem ser desenhadas de forma robusta, aderindo às boas práticas, para facilitar identificação e acesso aos recursos, ao mesmo tempo que tornam a API mais legível e fácil de usar.

Modelo de Maturidade de Richardson

Modelo de maturidade de Richardson (nível 2)

- **Nível 0:** Chamadas simples sem estrutura definida.
- **Nível 1:** Recursos identificados por URLs.
- **Nível 2:** Uso correto de métodos HTTP e códigos de status para ações específicas.
- **Nível 3:** HATEOAS (Hypermedia as the Engine of Application State).

Exemplos em cada nível

1. **Nível 0:** Chamadas simples sem estrutura definida

Neste nível, as APIs apenas utilizam um único endpoint sem diferenciar recursos ou métodos HTTP. Todas as requisições são feitas para a mesma URL e, normalmente, a ação ou tipo de operação é passada via query parameters, ou no corpo da requisição.

Exemplo:

```
curl -X POST http://api.example.com/api \
-H "Content-Type: application/json" \
-d '{"action": "getAllUsers"}'
```

- O endpoint `/api` é genérico.
- A operação é identificada pelo atributo `"action"` no corpo da requisição.
- Sem uso claro dos métodos HTTP para determinar a ação.

Exemplos em cada nível

2. **Nível 1:** Recursos identificados por URLs

Agora, recursos são organizados por URLs específicas que identificam os itens acessados. Porém, ainda sem usar adequadamente os métodos HTTP.

Exemplo:

```
curl http://api.example.com/users
```

- Recursos como `users` estão claramente definidos por sua URL (`/users` para listar os usuários).
- No entanto, todas as operações ainda podem ser feitas com o mesmo método HTTP, sem distinção.

Exemplos em cada nível

3. **Nível 2:** Uso correto de métodos HTTP e códigos de status

Neste nível, existe uma diferenciação clara das operações com base nos métodos HTTP usados (como GET, POST, PUT, DELETE), além da adoção de códigos de status apropriados para as respostas.

Exemplo:

- **Obter lista de usuários:**

```
curl -X GET http://api.example.com/users
```

- **Criar novo usuário:**

```
curl -X POST http://api.example.com/users \
  -H "Content-Type: application/json" \
  -d '{"name": "John Doe", "email": "john.doe@email.com"}'
```


Exemplos em cada nível

3. **Nível 2:** Uso correto de métodos HTTP e códigos de status

Neste nível, existe uma diferenciação clara das operações com base nos métodos HTTP usados (como GET, POST, PUT, DELETE), além da adoção de códigos de status apropriados para as respostas.

Exemplo:

- **Atualizar usuário:**

```
curl -X PUT http://api.example.com/users/1 \  
-H "Content-Type: application/json" \  
-d '{"name": "Jane Doe"}'
```

- **Excluir um usuário:**

```
curl -X DELETE http://api.example.com/users/1
```

Exemplos em cada nível

3. **Nível 2:** Uso correto de métodos HTTP e códigos de status

Neste nível, existe uma diferenciação clara das operações com base nos métodos HTTP usados (como GET, POST, PUT, DELETE), além da adoção de códigos de status apropriados para as respostas.

Exemplo:

- **Uso integrado de códigos de status:**
 - 200 (OK) para sucesso ao obter informações,
 - 201 (Created) ao criar um novo recurso,
 - 404 (Not Found) se o recurso não for encontrado,
 - etc.

Exemplos em cada nível

4. **Nível 3:** HATEOAS (*Hypermedia as the Engine of Application State*)

No último nível, além de organizar os recursos e métodos, adiciona-se a navegação hipertextual (HATEOAS). As respostas do servidor contêm links que guiam o cliente para outras interações possíveis com a API.

Modelo de maturidade de Richardson

Programação Web II

Exemplos em cada nível

4. **Nível 3:** HATEOAS (*Hypermedia as the Engine of Application State*)

Exemplo:

Resposta para **GET** `/users/1`:

A resposta inclui links no atributo `_links`, indicando claramente os próximos passos possíveis:

- Atualizar (PUT),
- Deletar (DELETE),
- Ou consultar o próprio recurso adicionalmente (self).

Esse modelo simplifica a interação entre cliente e servidor, pois o cliente conhece as ações disponíveis sem a necessidade de documentação extra.

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john.doe@email.com",
  "_links": {
    "self": {
      "href": "http://api.example.com/users/1"
    },
    "update": {
      "href": "http://api.example.com/users/1",
      "method": "PUT"
    },
    "delete": {
      "href": "http://api.example.com/users/1",
      "method": "DELETE"
    }
  }
}
```


Benefícios de APIs REST

Benefícios de APIs REST

Programação Web II

- Simplicidade de implementação.
- Alta escalabilidade.
- Compatibilidade com diferentes plataformas.
- Facilidade de integração com microsserviços.

Exercícios

Exercício 1: Integração Java com API de Produtos

Contexto: Você irá desenvolver um sistema Java que consulta produtos de uma API pública.

Link: [DummyJSON API](#)

Exercício 1: Integração Java com API de Produtos

Parte 1 – Primeiro contato com a API

API utilizada: <https://dummyjson.com/products>

Crie um programa Java que:

- faça uma requisição GET
- conecte na API
- exiba a resposta completa

Utilize:

- URL
- HttpURLConnection
- Scanner

Saída esperada:

```
{  
  "products": [ ... ]  
}
```

Exercício 1: Integração Java com API de Produtos

Parte 2 – Exibir apenas algumas informações

API utilizada: <https://dummyjson.com/products>

O programa deverá mostrar apenas:

- título
- preço
- categoria

Exemplo esperado

```
Produto: iPhone 13  
Preço: 899  
Categoria: smartphones
```

Exercício 1: Integração Java com API de Produtos

Parte 3 – Criar classe Produto

Crie uma classe: Produto

Com atributos: titulo, preco, categoria

Exercício 1: Integração Java com API de Produtos

Parte 4 – Busca dinâmica

O usuário poderá pesquisar produtos.

Exemplo:

Digite produto:

phone

Programa monta URL:

`https://dummyjson.com/products/search?q=phone`

Exercício 1: Integração Java com API de Produtos

Parte 5 – Tratamento de erros

Seu programa deve tratar:

- internet fora
- produto inexistente
- erro da API

Desafio Extra

Criar menu:

```
1 - Buscar produtos
2 - Listar produtos
3 - Sair
```

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Contexto: Você foi contratado para desenvolver um sistema simples de consulta de endereços utilizando a API pública ViaCEP.

Sua aplicação deverá:

- receber um CEP digitado pelo usuário
- realizar uma requisição HTTP
- consumir a API ViaCEP
- exibir os dados formatados
- tratar possíveis erros

API utilizada: <https://viacep.com.br/ws/01001000/json/>

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Resultado esperado

```
=====
CONSULTA DE CEP
=====
```

```
Digite o CEP:
```

```
01001000
```

```
CEP: 01001-000
```

```
Logradouro: Praça da Sé
```

```
Bairro: Sé
```

```
Cidade: São Paulo
```

```
Estado: SP
```

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Requisitos obrigatórios:

O sistema deve possuir:

- Scanner
- Classes
- Objetos
- Métodos
- Consumo HTTP
- Tratamento de erros
- Organização em classes

Estrutura esperada do projeto

Sugestão:

```
src/  
├── Main.java  
├── CepService.java  
└── Endereco.java
```


Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 1 – Criando a classe Endereco

Crie uma classe chamada: Endereco

A classe deve possuir:

- cep
- logadouro
- bairro
- localidade
- uf

Requisitos

A classe deve possuir:

- atributos privados
- getters e setters
- método construtor
- método toString()

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 2 – Criando conexão HTTP

Crie uma classe chamada: CepService

Responsabilidade

Essa classe deverá:

- conectar na API
- fazer requisição GET
- retornar resposta da API

URL dinâmica

A URL deve ser montada usando o CEP digitado pelo usuário.

Exemplo: `https://viacep.com.br/ws/01001000/json/`

Requisitos

Utilize:

- URL
- HttpURLConnection
- Scanner

Dica: `URL url = new URL(enderecoApi);`

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 3 – Exibir resposta da API

O sistema deve:

- imprimir o JSON retornado
- validar se a conexão funcionou

Exemplo esperado

```
{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "uf": "SP"
}
```

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 4 – Melhorando saída para usuário

Agora o sistema NÃO deve mais exibir o JSON completo.

Ele deve exibir apenas:

```
CEP:  
Logradouro:  
Bairro:  
Cidade:  
Estado:
```

Objetivo: Transformar a resposta da API em algo amigável.

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 5 – Entrada dinâmica do usuário

O sistema deve solicitar:

Regras: O usuário poderá digitar qualquer CEP válido.

Exemplo: 13010010

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 6 – Tratamento de erros

Seu sistema deve tratar:

- **CEP inválido:** 11111111
- **Usuário digitando letras:** ABC123
- **CEP incompleto:** 123
- **Falha de conexão:** Caso a API esteja indisponível.

Exemplo esperado:

CEP inválido! OU Erro ao conectar com a API.

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 7 – Criando Menu Interativo

O sistema deverá possuir menu:

```
=====
1 - Consultar CEP
2 - Sair
=====
```

Regras:

- **opção 1** → consulta CEP
- **opção 2** → encerra sistema
- **opção inválida** → mostrar mensagem

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Parte 8 – Refatoração e Organização

Revise seu código e verifique:

- nomes claros
- métodos organizados
- responsabilidade correta das classes
- tratamento de erros
- indentação

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Desafio Extra 1 – Histórico de consultas

Salvar CEPs pesquisados em uma lista.

Exemplo:

```
CEPs pesquisados:  
01001000  
13010010  
30140071
```

Exercício 2: Sistema de Consulta CEP com Java + ViaCEP

Desafio Extra 2 – Consulta infinita

Permitir:

- múltiplas consultas
- sem reiniciar programa

Desafio Extra 3 – Melhorar visual

Adicionar:

- separadores
- títulos
- mensagens amigáveis

Desafio Extra 4 – Validar apenas números

O sistema deve impedir: **ABC12345**

Obrigada